

```

•   pLabel3->SetTextVerticalAlignment(ALIGNMENT _ MIDDLE);
•   pLabel3->SetTextHorizontalAlignment(ALIGNMENT _ LEFT);
•   AddControl(*pLabel3);
•
•   // Similarly, create Label4 for "Without a privilege"
•
•   // Create a button
•   Button *pButton1 = new Button();
•   pButton1->Construct(Rectangle(55, 190, 170, 70));
•   pButton1->SetText(L"Test");
•   pButton1->SetActionId(ID _ BUTTON1);
•   pButton1->AddActionEventListener(*this);
•   AddControl(*pButton1);
•
•   // Similarly, create Button2
•
•   return r;
• }

```

2. The `UpdatePanel()` method is used to refresh the labels every time the panel is selected.

```

void PrivilegePanel::UpdatePanel(void)
{
•   Label *pLabel = static _ cast<Label
•   *>(GetControl(L"Label1"));
•   if(pLabel != null)
•       pLabel->SetText(L"Result");
•
•   // Label2 is handled in the same way
•
• }

```

3. The `OnActionPerformed()` event handler defines cases for the two action IDs mentioned in the `PrivilegePanel.h` file.

The two methods that require privileges are `SystemInfo::GetValue()` and `WebHistory::Construct()`, called because the `pButton1` and `pButton2` buttons have been clicked, respectively.

Only the `SystemInfo::GetValue()` method is successful, since its required privilege has been set in the manifest file.

```

void PrivilegePanel::OnActionPerformed(const
Osp::Ui::Control& source, int actionId)

```